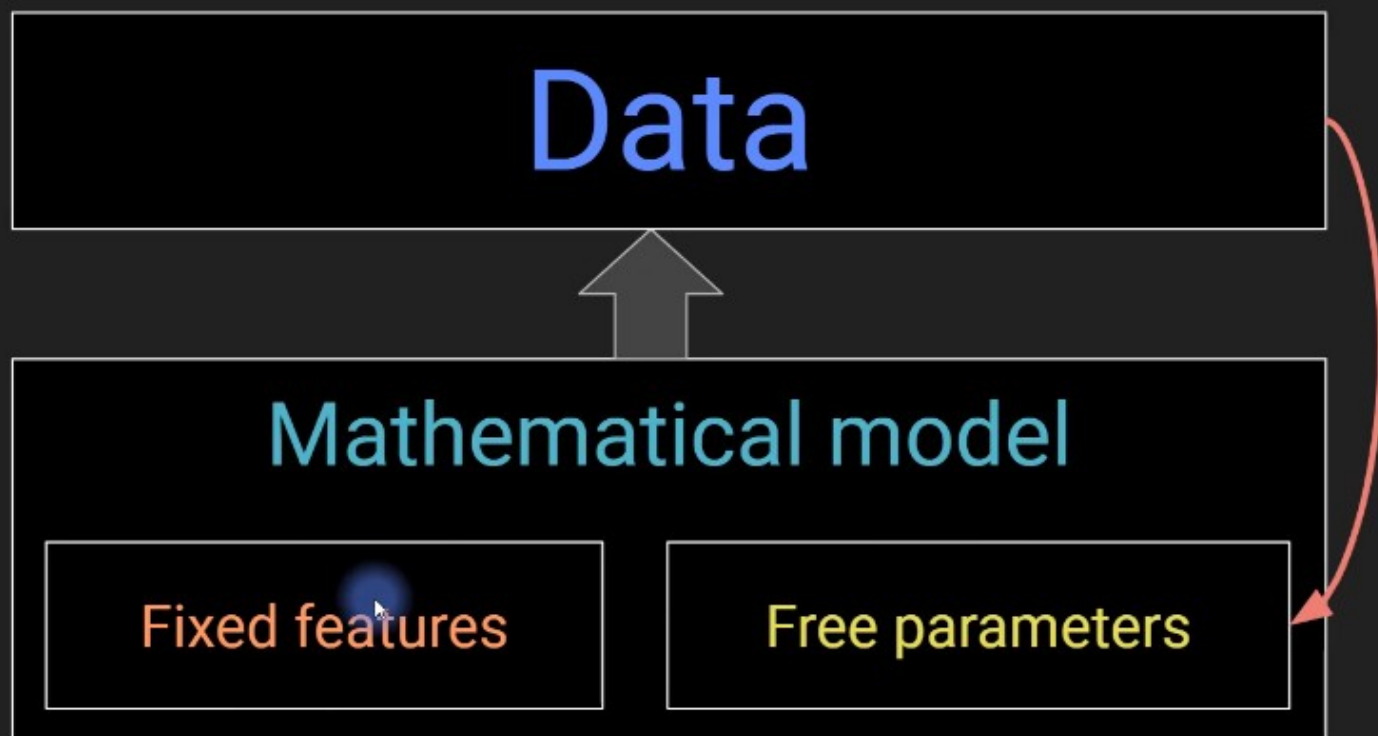


Introduction to GLM / regression

In this video, you will learn

- What “GLM” stands for (hint: general linear model) and why it’s so *general*.
- The formula for the GLM / regression.
- The steps to setting up a GLM.

The model and the data



ANOVA vs. regression

Use an ANOVA when ALL IVs are discrete (usually categorical).

Use a regression when at least some IVs are continuous.

The five steps to model-fitting

- Step 1: Define the equation(s) underlying the model.
- Step 2: Map the data into the model equations.
- Step 3: Convert the equations into a matrix-vector equation.
- Step 4: Compute the parameters.
- Step 5: Statistical evaluation of the model.

The five steps to model-fitting

Step 1: Define the equation(s) underlying the model.

$$h = \beta_0 + \beta_1 s + \beta_2 p + \beta_3 n$$

h = height

β_0 = intercept (average height when all other parameters are 0)

s = sex (m/f), dummy-coded

p = parents' height

n = childhood nutrition

The five steps to model-fitting

Step 1: Define the equation(s) underlying the model.

Step 2: Map the data into the model equations.

Person ID	Height (cm)	Sex	Parents' height (cm)	Childhood nutrition
1	180	M	175	8
2	170	F	172	6
...	...	...	...	...
10	176	F	189	7

The five steps to model-fitting

Step 1: Define the equation(s) underlying the model.

Step 2: Map the data into the model equations.

$$h = \beta_0 + \beta_1 s + \beta_2 p + \beta_3 n$$



$$180 = \beta_0 1 + \beta_1 1 + \beta_2 175 + \beta_3 8$$

$$170 = \beta_0 1 + \beta_1 0 + \beta_2 172 + \beta_3 6$$

⋮

$$176 = \beta_0 1 + \beta_1 0 + \beta_2 189 + \beta_3 7$$

The five steps to model-fitting

Step 1: Define the equation(s) underlying the model.

Step 2: Map the data into the model equations.

Step 3: Convert the equations into a matrix-vector equation.

$$\left. \begin{aligned} 180 &= \beta_0 1 + \beta_1 1 + \beta_2 175 + \beta_3 8 \\ 170 &= \beta_0 1 + \beta_1 0 + \beta_2 172 + \beta_3 6 \\ &\vdots \\ 176 &= \beta_0 1 + \beta_1 0 + \beta_2 189 + \beta_3 7 \end{aligned} \right\} \begin{pmatrix} 1 & 1 & 175 & 8 \\ 1 & 0 & 172 & 6 \\ \vdots & & & \\ 1 & 0 & 189 & 7 \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \beta_3 \end{pmatrix} = \begin{pmatrix} 180 \\ 170 \\ \vdots \\ 176 \end{pmatrix} \quad \mathbf{X}\boldsymbol{\beta} = \mathbf{y}$$

The five steps to model-fitting

Step 1: Define the equation(s) underlying the model.

Step 2: Map the data into the model equations.

Step 3: Convert the equations into a matrix-vector equation.

Step 4: Compute the parameters.

Next lecture...

The five steps to model-fitting

Step 1: Define the equation(s) underlying the model.

Step 2: Map the data into the model equations.

Step 3: Convert the equations into a matrix-vector equation.

Step 4: Compute the parameters.

Step 5: Statistical evaluation of the model.

Two questions:

1. Is the model a good fit to the data?
2. Are individual β coefficients statistically significant?

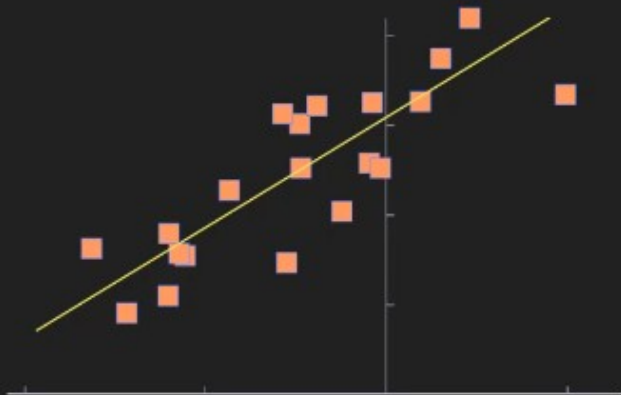
$$h = \beta_0 + \beta_1 s + \beta_2 p + \beta_3 n$$

The intercept

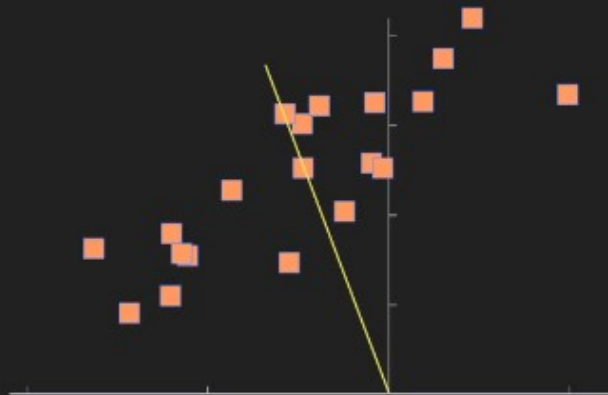
The intercept term is usually not interpreted and usually not very interesting.

But it needs to be in the model because it captures the offset of the data when all other predictors are set to 0.

With intercept



No intercept



Statistics terminology for GLM

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta}$$

General linear model (GLM)

Sometimes $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$

$\mathbf{X}$

Design matrix (columns = independent variables, predictors, regressors)

$\boldsymbol{\beta}$

Regression coefficients or beta parameters

$\mathbf{y}$

Dependent variable or outcome measure or data

General **LINEAR** model

GLMs (ANOVA, regression, correlation) are linear models.

Linear means scalar-multiplication and addition *of the regressors*.

Log, square root, powers, trigonometry etc., are all nonlinear operations.

The data can have nonlinearities; only the parameters must be linear in the model.

Nonlinear data are often linearized to facilitate interpretation.

General **LINEAR** model

$$h = \beta_0 + \beta_1 s + (\beta_1 / \ln(\beta_2))^3 s p + \beta_2 p + \beta_3 n$$

Unacceptable!

Acceptable!
(interaction)

NON
LINEAR

Least-squares solution to the GLM

In this video, you will learn

- How to solve the GLM equation to obtain the regression coefficients.
- (Warning: The math requires some linear algebra background, but is not crucial to understanding GLM.)

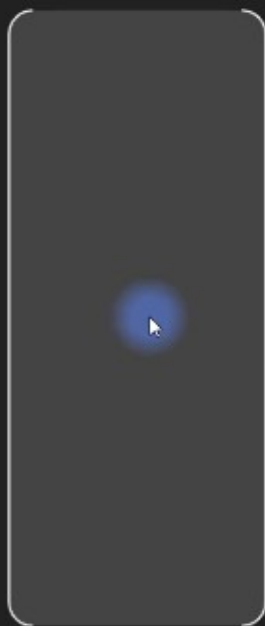
Linear least-squares via left inverse

$$\underbrace{(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T}_{\text{Left inverse}} \mathbf{X} \boldsymbol{\beta} = \underbrace{(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T}_{\text{Left inverse}} \mathbf{y}$$

$$\boldsymbol{\beta}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Conditions on X for left inverse to exist

$$\beta = (X^T X)^{-1} X^T y$$



- 1) Tall matrix
- 2) "Independent" IVs

Conditions on X for left inverse to exist

$$\beta = (X^T X)^{-1} X^T y$$

$$\begin{aligned} 180 &= \beta_0 + \beta_1 1 + \beta_2 175 + \beta_3 8 \\ 170 &= \beta_0 + \beta_1 0 + \beta_2 172 + \beta_3 6 \\ &\vdots \\ 176 &= \beta_0 + \beta_1 0 + \beta_2 189 + \beta_3 7 \end{aligned} \left\{ \begin{array}{ccc} 175 & 175 & 8 \\ 172 & 172 & 6 \\ \vdots & \vdots & \vdots \\ 189 & 189 & 7 \end{array} \right\} \text{Rank}=2$$

“Multicollinearity”

Conditions on X for left inverse to exist

$$\beta = (X^T X)^{-1} X^T y$$

$$\begin{aligned} 180 &= \beta_0 + \beta_1 1 + \beta_2 175 + \beta_3 8 \\ 170 &= \beta_0 + \beta_1 0 + \beta_2 172 + \beta_3 6 \\ &\vdots \\ 176 &= \beta_0 + \beta_1 0 + \beta_2 189 + \beta_3 7 \end{aligned} \left\{ \begin{bmatrix} 1 & 175 & 8 \\ 0 & 172 & 6 \\ \vdots & \vdots & \vdots \\ 0 & 189 & 7 \end{bmatrix} \right.$$

Rank=3

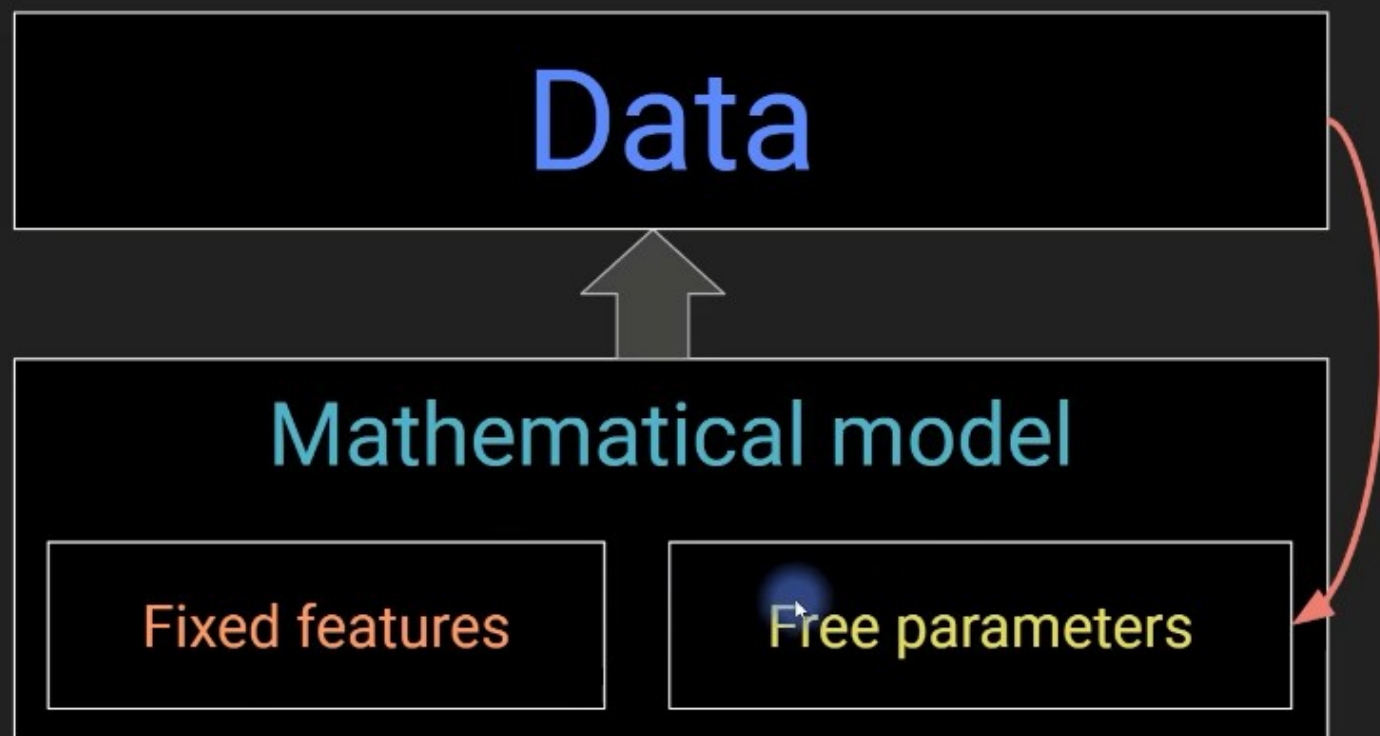
"A-OK!"

Evaluating regression models: R^2 and F

In this video, you will learn

- How to evaluate the overall fit of the model to the data.
- The degrees of freedom of a regression model.
- How to evaluate the statistical significance of each regression parameter (β).

The model and the data



The model and the residuals

$$y = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k + \epsilon$$

$$\epsilon = y - (\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)$$

$$\hat{y} = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k$$

$$\epsilon = y - \hat{y}$$

Evaluating a model fit with R^2

$$R^2 = 1 - \frac{SS_{\epsilon}}{SS_{\text{Total}}} = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

R^2 close to 1 means the model fits the data well.

R^2 close to 0 means the model fits the data poorly.

R^2 below 0 is most likely due to an error.

There is no cut-off for a “good” R^2 !

R^2 is often used in model comparisons.

Evaluating model statistical significance with F

$$H_0: \beta_{1-k} = 0$$

$$y = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k + \epsilon$$

$$H_A: \text{At least one } \beta \neq 0$$

$$y = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k + \epsilon$$

Evaluating model statistical significance with F

$$SS_{\epsilon} = \sum (y_i - \hat{y})^2$$

$$SS_{\text{Model}} = \sum (\hat{y}_i - \bar{y})^2$$

$$F(k - 1, N - k) = \frac{SS_{\text{Model}} / (k - 1)}{SS_{\epsilon} / (N - k)}$$

Significance of individual β coefficients

$$t_{N-k} = \frac{\beta_j}{s_\beta} = \frac{\beta_j}{\sqrt{\frac{SS_\epsilon}{N-k} (X^T X)^{-1}_{jj}}}$$

Simple regression

In this video, you will learn

- More about performing regression analyses and interpreting the results.
- What “simple regression” means.
- The importance of sample size for statistics.

What is a “simple regression”?

Simple regression has one DV and one IV
(technically two IVs including the intercept).

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i$$

i^{th} data value

intercept

Coefficient
and IV

i^{th} error
(residual)

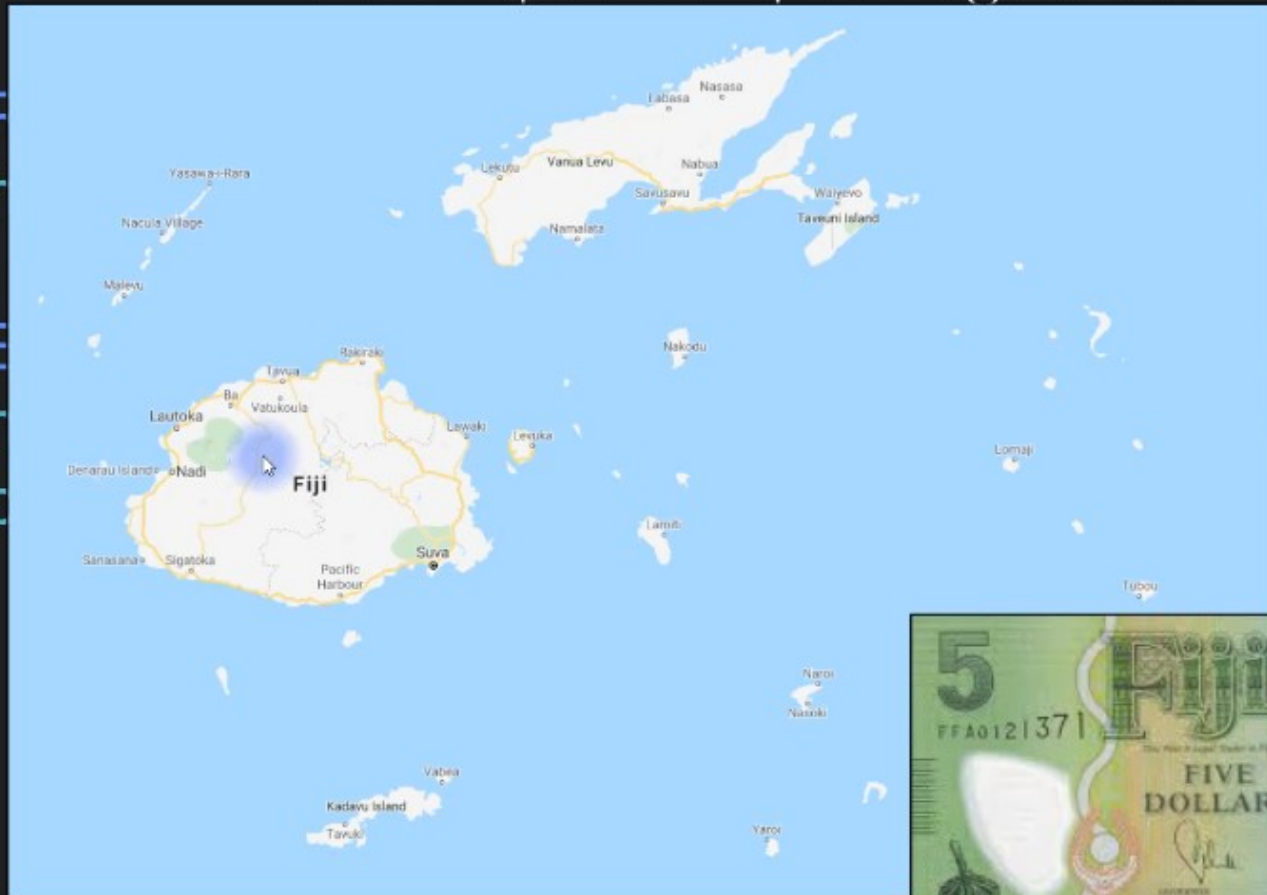
$$\epsilon_i = y_i - (\beta_0 + \beta_1 x_i)$$

Example simple regression

Research question: Does the amount of sleep affect how much money people spend on food shopping?

Experiment: Ask random people ($N=10$) at a food market to report the number of hours of sleep the previous night, and their total shopping bill.

Example simple regression



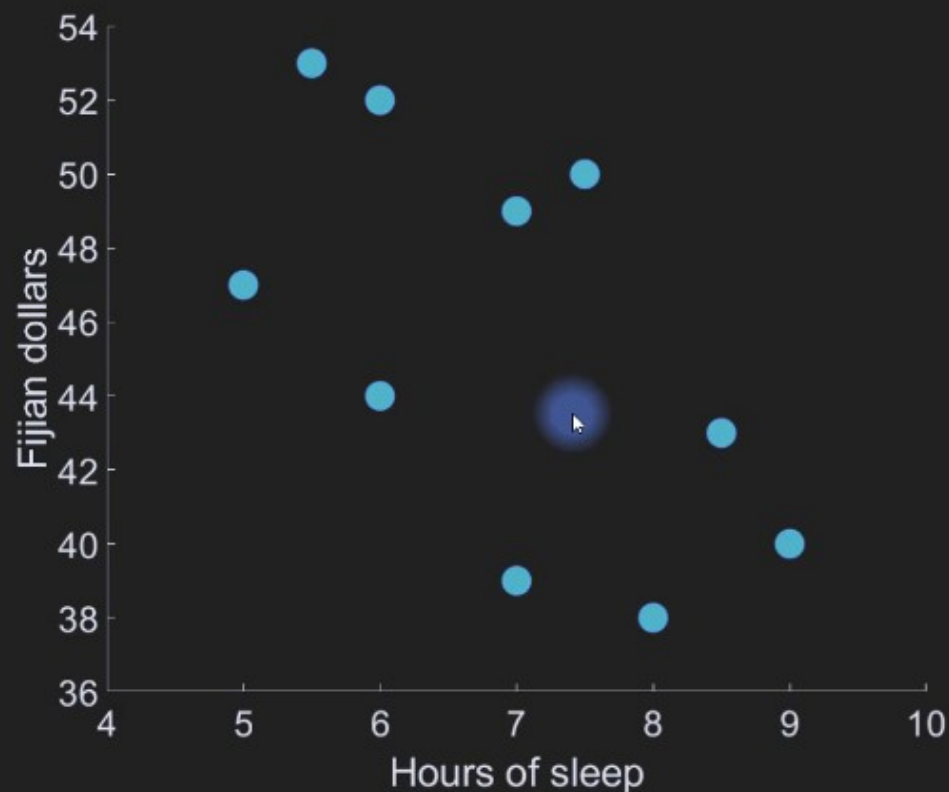
affect
ping?

ood
the



The data

Sleep	Fiji\$
5	47
5.5	53
6	52
6	44
7	39
7	49
7.5	50
8	38
8.5	43
9	40



The model equation

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i$$

Bill for i^{th}
person

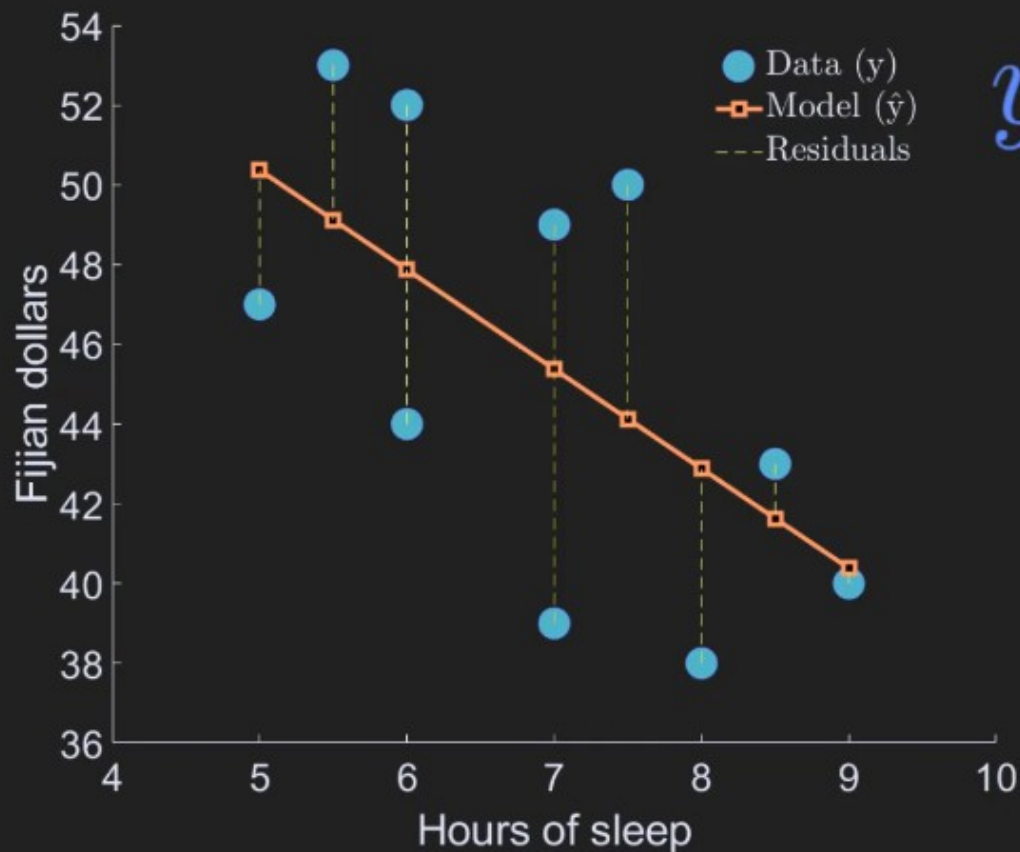
Intercept

Hours of
sleep

Residual

$$\mathbf{X}\boldsymbol{\beta} = \mathbf{y} \Rightarrow \begin{bmatrix} 1 & 5 \\ 1 & 5.5 \\ 1 & 6 \\ 1 & 6 \\ 1 & 7 \\ 1 & 7 \\ 1 & 7.5 \\ 1 & 8 \\ 1 & 8.5 \\ 1 & 9 \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} = \begin{bmatrix} 47 \\ 53 \\ 52 \\ 44 \\ 39 \\ 49 \\ 50 \\ 38 \\ 43 \\ 40 \end{bmatrix} \Rightarrow \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} = \begin{bmatrix} 63 \\ -2.5 \end{bmatrix}$$

Actual and predicted data



$$y_i = 63 - 2.5x_i + \epsilon_i$$

$$R^2 = .36$$

$$F(1, 8) = 4.54$$

$$p = .066$$

Conclusion: The model is not a statistically significant fit to the data. This may be due to small sample size ($N=40 \rightarrow p=.039$).

```

import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats

## example: effects of sleep on food spending
sleepHours = [5, 5.5, 6, 6, 7, 7, 7.5, 8, 8.5, 9]
dollars = [47, 53, 52, 44, 39, 49, 50, 38, 43, 40]
# start by showing the data
plt.plot(sleepHours,dollars,'ko',markerfacecolor='k')
plt.xlabel('Hours of sleep')
plt.ylabel('Fijian dollars')
plt.show()

## "manual" regression via least-squares fitting
# create the design matrix
desmat = np.vstack((np.ones(10),sleepHours)).T
print(desmat)

# compute the beta parameters (regression coefficients)
beta = np.linalg.lstsq(desmat,dollars,rcond=None)[0]
print(beta)

# predicted data values
yHat = desmat@beta

## show the predicted results on top of the "real" data
# show previous data
plt.plot(sleepHours,dollars,'ko',markerfacecolor='k')

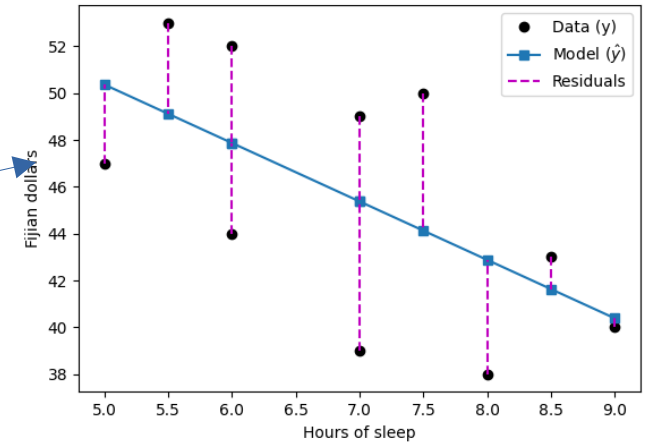
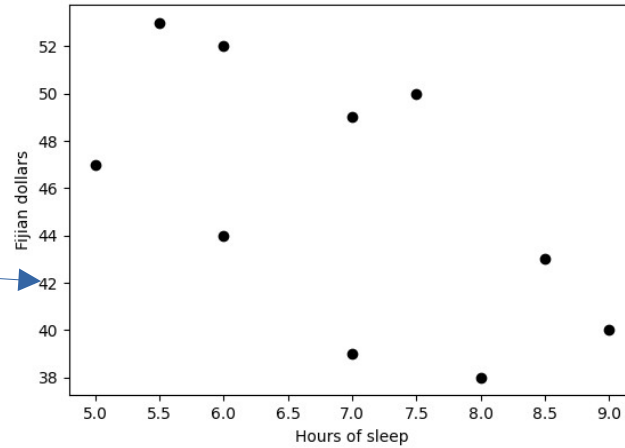
# predicted results
plt.plot(sleepHours,yHat,'s-')

# show the residuals
for i in range(10):
    plt.plot([sleepHours[i],sleepHours[i]],[dollars[i], yHat[i]],'m--')

plt.legend(('Data (y)','Model ( $\hat{y}$ )','Residuals'))
plt.xlabel('Hours of sleep')
plt.ylabel('Fijian dollars')
plt.show()

## now with scipy
slope,intercept,r,p,std_err = stats.linregress(sleepHours,dollars)
print(intercept,slope)
print(beta)

```



```

[[[1. 5. ]
 [1. 5.5]
 [1. 6. ]
 [1. 6. ]
 [1. 7. ]
 [1. 7. ]
 [1. 7.5]
 [1. 8. ]
 [1. 8.5]
 [1. 9. ]]]
[62.84737679 -2.49602544]
62.84737678855326 -2.4960254372019075
[62.84737679 -2.49602544]

```

Multiple regression

In this video, you will learn

- More about performing regression analyses and interpreting the results.
- An example of a multiple regression.
- How to construct an interaction term in a multiple regression model.
- How to interpret a regression table.

Previous example of multiple regression

The five steps to model-fitting

Step 1: Define the equation(s) underlying the model.

$$h = \beta_0 + \beta_1 s + \beta_2 p + \beta_3 n$$

h = height

β_0 = intercept (average height when all other parameters are 0)

s = sex (m/f), dummy-coded

p = parents' height

n = childhood nutrition

Another example

Research question: Do students' grade on a stats exam depend on sleep (s), hours studied (h), and the interaction between them? ($N=30$)

$$y = \beta_0 + \beta_1 s + \beta_2 h + \beta_3 (s \times h) + \epsilon$$

Regression table

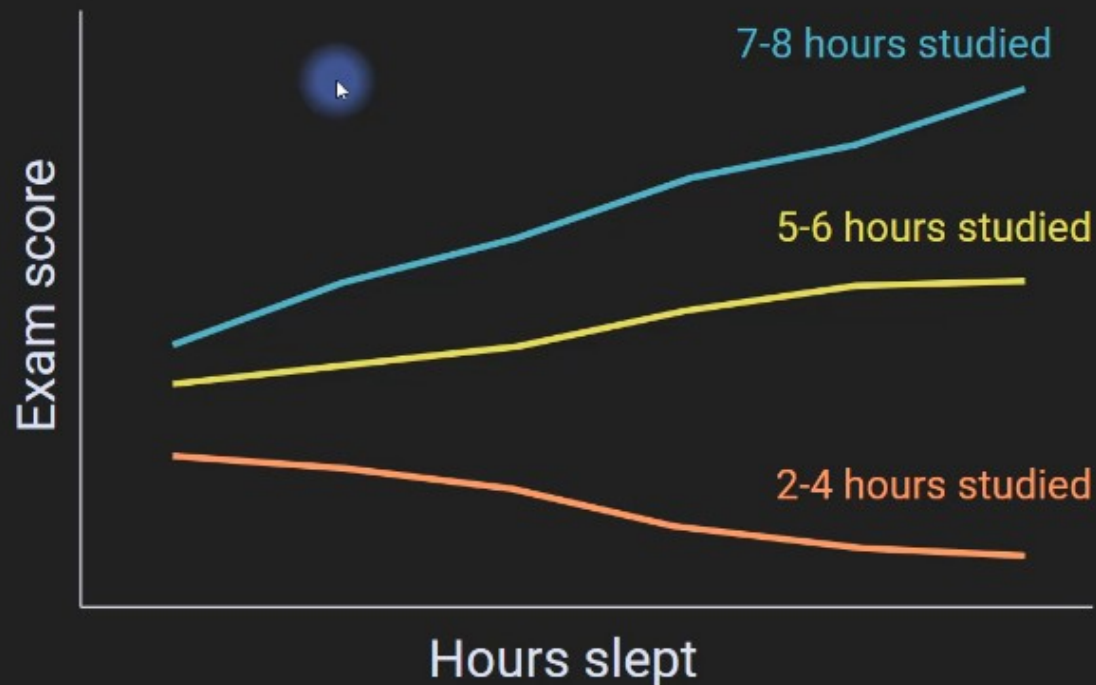
Source of variance	β coeff.	Std. error	t-value (df=??)	p-value
Constant	60	.3	200	.000
Study hours	4.5	1.2	3.75	.001
Sleep hours	.8	2.4	.33	.373
Study $\times$ sleep	5.4	.8	6.75	.000

$$df = N - k = 30 - 4 = 26$$

Visualizing the results

Conclusions

- Main effect of study (more study -> higher score)
- No main effect of hours slept.
- Significant interaction (more study is better with enough sleep).



Higher-order interactions?

$$\begin{aligned} y = & \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 \\ & + \beta_4 (x_1 \times x_2) + \beta_5 (x_1 \times x_3) + \beta_6 (x_2 \times x_3) \\ & + \beta_7 (x_1 \times x_2 \times x_3) + \epsilon \end{aligned}$$

Conclusion: Higher-order interaction terms are mathematically fine, but become difficult to interpret (easier if at least one variable is binary). Use only when necessary!

Precise interpretation of β in a multiple regression

$$y = \beta_0 + \beta_1 s + \beta_2 h + \beta_3 (s \times h) + \epsilon$$

Interpretation: β_2 reflects the effect of a change in h on y when all other variables are held constant.

Standardized regression coefficients

In this video, you will learn

- The difference between unstandardized and standardized regression coefficients.
- How to standardize β weights.

The scales of the β 's

$$y = \beta_0 + \beta_1 s + \beta_2 h + \beta_3 (s \times h) + \epsilon$$

Beta parameters and IV scales

	Hours	Minutes	Seconds	Calories
β_1	.167	10	600	
β_2				.0051

Difficulties with interpreting β 's

Unstandardized β coefficients change depending on the scale of the IV.

Unstandardized β coefficients can be difficult (or impossible) to compare across variables (and studies...).

These difficulties motivate a standardization of β coefficients.

The scales of the β 's

$$y = \beta_0 + \boxed{\beta_1 s} + \beta_2 h + \beta_3 (s \times h) + \epsilon$$

Standardized beta parameters

	Hours	Minutes	Seconds	Calories
β_1	.6	.6	.6	
β_2				.8

The scales of the β 's

Unstandardized β coefficients reflect the scales of the data (IV and DV). This can facilitate interpretation but can also stymie comparisons across variables or models.

Standardized β coefficients are in standard deviation units, unrelated to the scales of the data.

Both are correct and neither is better; sometimes one is more natural or easier to interpret than the other.

Importantly, standardization has no effect on the statistics!

How to standardize regressors

Basic idea: normalize β so that its variance is 1.

Method 1: z-normalize DV and IVs before the regression. All β 's will be in the units of the data, which are already standard deviation units.

Method 2: Scale the unstandardized β by the standard deviations of the IV and corresponding DV.

$$b_k = \beta_k \frac{s_{x_k}}{s_y}$$

Interpreting standardized β 's

$$y = \beta_0 + \beta_1 s + \boxed{\beta_2 h} + \beta_3 (s \times h) + \epsilon$$

Interpretation: β_2 reflects the effect of a one-standard deviation change in h on standard deviation changes in y , when all other variables are held constant.

```

import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats
from sklearn.linear_model import LinearRegression
import statsmodels.api as sm

## example: effects of sleep and study hours on exam scores
### create the data
exam_scores = []
for ei in range(5):
    exam_scores = np.hstack((exam_scores, 60*np.ones(6)+np.linspace(-1,5,6)*ei))

hours_studied = np.tile(np.linspace(2,8,6),5)
ave_sleep_hrs = np.linspace(6,10,30)

## plot the data
### stratify by hours studied
# fewer than 4 hours studied
plotidx = hours_studied<4.1
plt.plot(ave_sleep_hrs[plotidx], exam_scores[plotidx], 'ko', markerfacecolor='k')

# 5-6 hours studied
plotidx = np.logical_and(hours_studied>4.9, hours_studied<6.1)
plt.plot(ave_sleep_hrs[plotidx], exam_scores[plotidx], 'ro', markerfacecolor='r')

# more than 6 hours
plotidx = hours_studied>6
plt.plot(ave_sleep_hrs[plotidx], exam_scores[plotidx], 'bo', markerfacecolor='b')

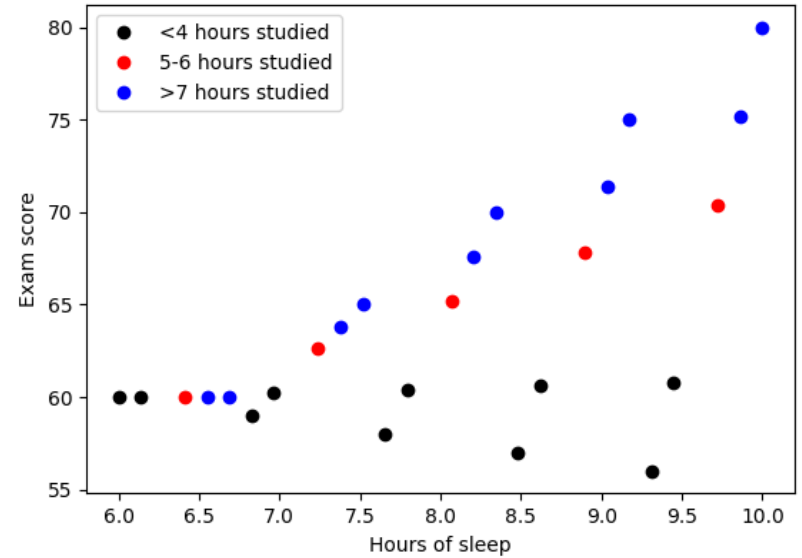
plt.xlabel('Hours of sleep')
plt.ylabel('Exam score')
plt.legend(['<4 hours studied', '5-6 hours studied', '>7 hours studied'])
plt.show()

## multiple regression
# build the design matrix
desmat = np.vstack((np.ones((30,)), ave_sleep_hrs, hours_studied, ave_sleep_hrs*hours_studied)).T
multireg = sm.OLS(endog=exam_scores, exog=desmat).fit()
multireg.summary()

# without the interaction term
multireg = sm.OLS(endog=exam_scores, exog=desmat[:,0:-1]).fit()
multireg.summary()

# inspect the correlations of the IVs
c=np.corrcoef(desmat[:,1:].T)
print(c)

```



```

from pandas.core import (
[[1.         0.19731231 0.49270769]
 [0.19731231 1.         0.94068915]
 [0.49270769 0.94068915 1.         ]]
atalet@uzgem:~/workspace/YZ EGITIM/ozet1

```


Polynomial regression model

In this video, you will learn

- The form of the polynomial regression.
- How to fit the coefficients of a polynomial regression.
- How to use the Bayes Information Criterion to determine the appropriate model order.

Reminder of polynomials

$$a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$

$$3 - 5x + \frac{5}{6}x^2 + 12.4x^3$$

$$2x + x^3$$

Polynomial regression

$$y = \beta_0 + \beta_1 x + \epsilon$$

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \epsilon$$

$\vdots$

$$y = \beta_0 x^0 + \beta_1 x^1 + \dots + \beta_k x^k + \epsilon$$

Polynomial regression

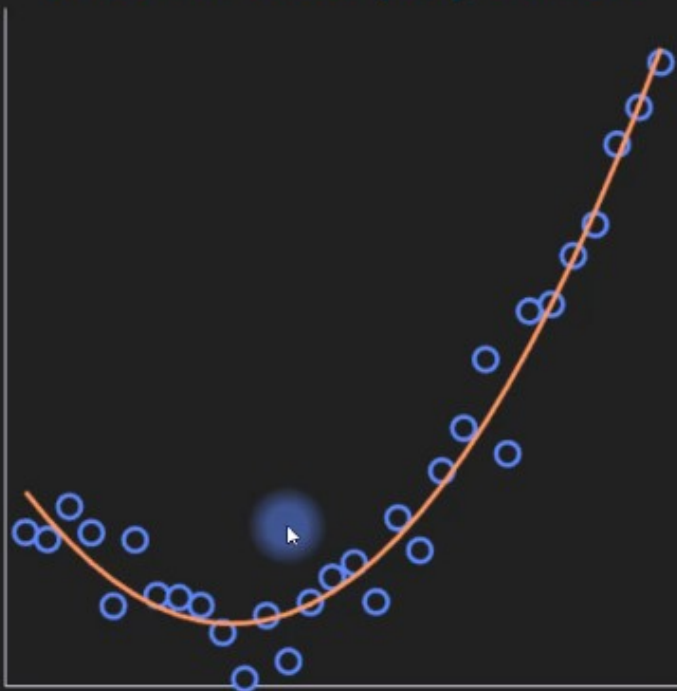
But how can we fit this kind of model with the nonlinearities?

The coefficients are all linear! This is a standard linear model!

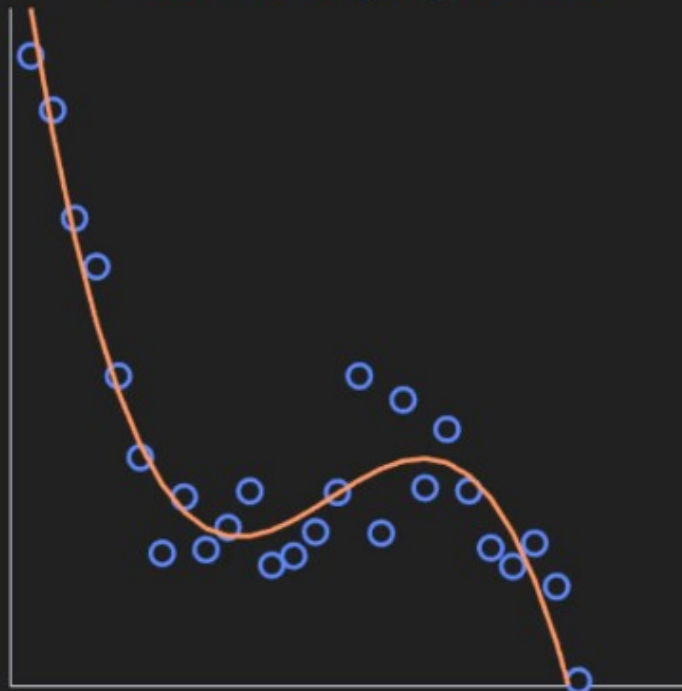
$$y = \beta_0 x^0 + \beta_1 x^1 + \dots + \beta_k x^k + \epsilon$$

When would you use a polynomial regression?

Second-order polynomial



Third-order polynomial



Polynomial regression

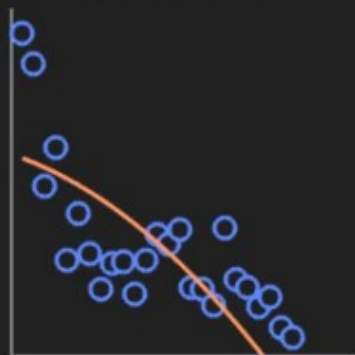
But how can we fit this kind of model with the nonlinearities?

The coefficients are all linear! This is a standard linear model!

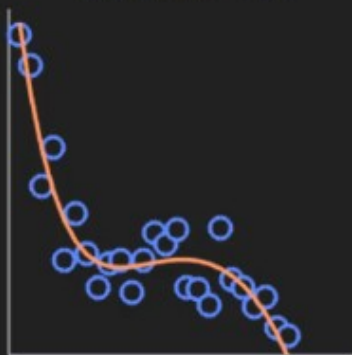
$$y = \beta_0 x^0 + \beta_1 x^1 + \dots + \beta_k x^k + \epsilon$$

Which order should you use?

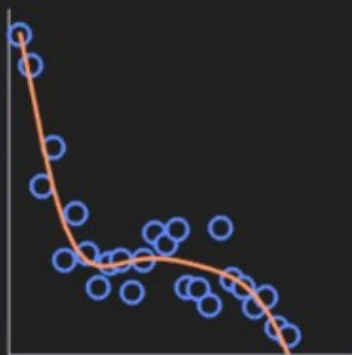
Order = 2



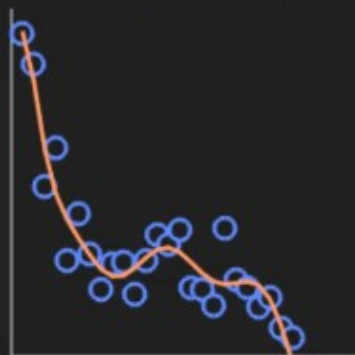
Order = 5



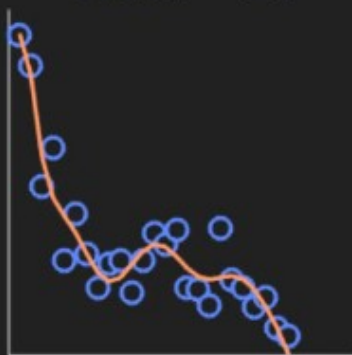
Order = 8



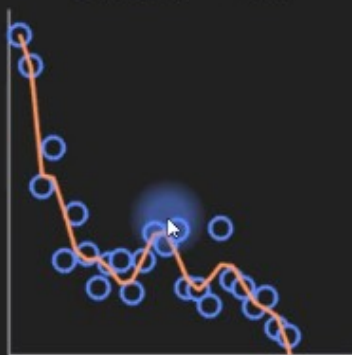
Order = 11



Order = 14

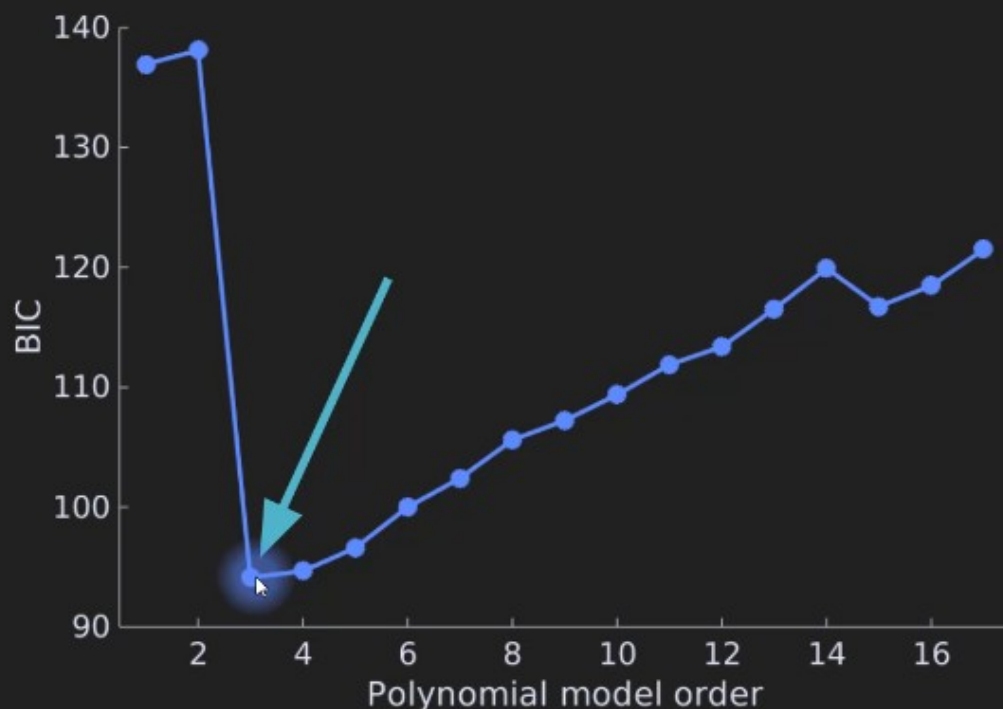


Order = 17



Model order selection, thanks to Bayes

$$\text{BIC}_k = n \ln(\text{SS}_\epsilon) + k \ln(n)$$



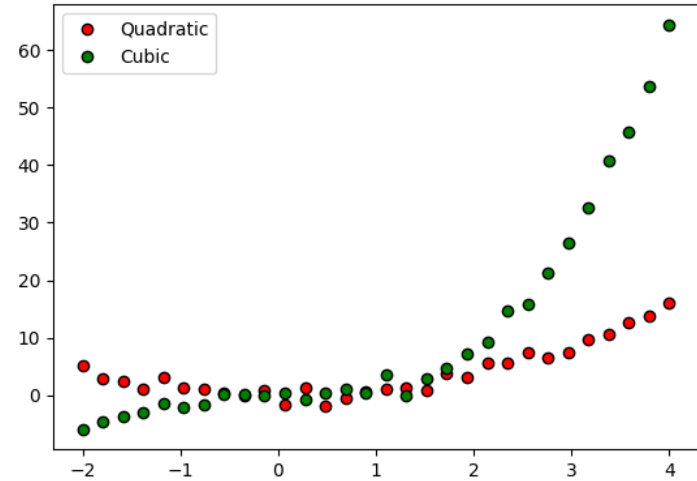
```
import numpy as np
import matplotlib.pyplot as plt
```

```
## generate the data
```

```
n = 30
x = np.linspace(-2,4,n)
y1 = x**2 + np.random.randn(n)
y2 = x**3 + np.random.randn(n)
```

```
# plot the data
```

```
plt.plot(x,y1,'ko',markerfacecolor='r')
plt.plot(x,y2,'ko',markerfacecolor='g')
plt.legend(('Quadratic','Cubic'))
plt.show()
```



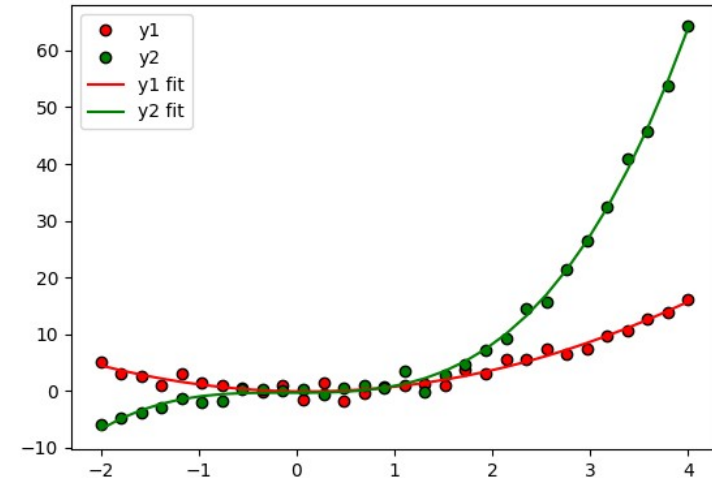
```
## now for a polynomial fit
```

```
# for y1
pterm = np.polyfit(x,y1,2)
yHat1 = np.polyval(pterm,x)
# for y2
```

```
pterm = np.polyfit(x,y2,3)
yHat2 = np.polyval(pterm,x)
```

```
# and all the plotting
```

```
plt.plot(x,y1,'ko',markerfacecolor='r',label='y1')
plt.plot(x,y2,'ko',markerfacecolor='g',label='y2')
plt.plot(x,yHat1,'r',label='y1 fit')
plt.plot(x,yHat2,'g',label='y2 fit')
plt.legend()
plt.show()
```



```

# compute R2
# compute R2 for several polynomial orders
orders = np.arange(1,6)

# output matrices
r2 = np.zeros((2,len(orders)))
sse = np.zeros((2,len(orders)))

# the loop!
for oi in range(len(orders)):
    # fit the model with oi terms
    pterms = np.polyfit(x,y1,orders[oi])
    yHat = np.polyval(pterms,x)
    # compute R2
    ss_eta = sum((y1-yHat)**2) # numerator
    ss_tot = sum((y1-np.mean(y1))**2) # denominator
    r2[0,oi] = 1 - ss_eta/ss_tot # R^2
    sse[0,oi] = ss_eta # store just the SSe for model comparison later
    ### repeat for y2
    pterms = np.polyfit(x,y2,orders[oi])
    yHat = np.polyval(pterms,x)
    ss_eta = sum((y2-yHat)**2)
    ss_tot = np.var(y2)*(n-1)
    r2[1,oi] = 1 - ss_eta/ss_tot
    sse[1,oi] = ss_eta

fig,ax = plt.subplots(2,1,figsize=(6,4))

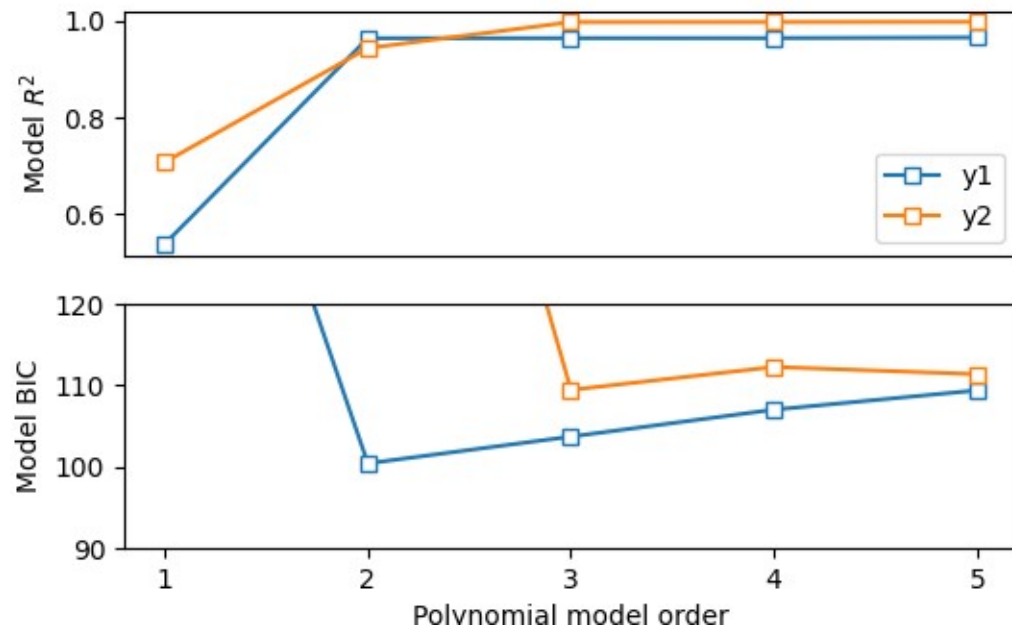
# plot the R2 results
ax[0].plot(orders,r2[0,:],'s-',markerfacecolor='w')
ax[0].plot(orders,r2[1,:],'s-',markerfacecolor='w')
ax[0].set_ylabel('Model $R^2$')
ax[0].set_xticks([])
ax[0].legend(('y1','y2'))

# compute the Bayes Information Criterion
bic = n*np.log(sse) + orders*np.log(n)
ax[1].plot(orders,bic[0,:],'s-',markerfacecolor='w')
ax[1].plot(orders,bic[1,:],'s-',markerfacecolor='w')
ax[1].set_xlabel('Polynomial model order')
ax[1].set_xticks(range(1,6))
ax[1].set_ylabel('Model BIC')

# optional zoom
ax[1].set_ylim([90,120])

plt.show()

```



Logistic regression

In this video, you will learn

- The difference between “normal” and logistic regression.
- When a regression analysis is logistic.
- How the logistic regression differs from the standard regression.

What is a logistic regression?

A logistic regression (a.k.a. binary logistic regression) has a binary DV (“logical”). Examples:

true / false

male / female

has tumor / doesn't have tumor

win / lose

Can be extended to any number of categorical outcomes (a.k.a. multinomial logistic regression):

cat / penguin / truck

<https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/>

What is the outcome of a logistic regression

A logistic regression does not classify; it returns probabilities of category membership.

Classification can be implemented using a threshold, e.g., $r > .5$.

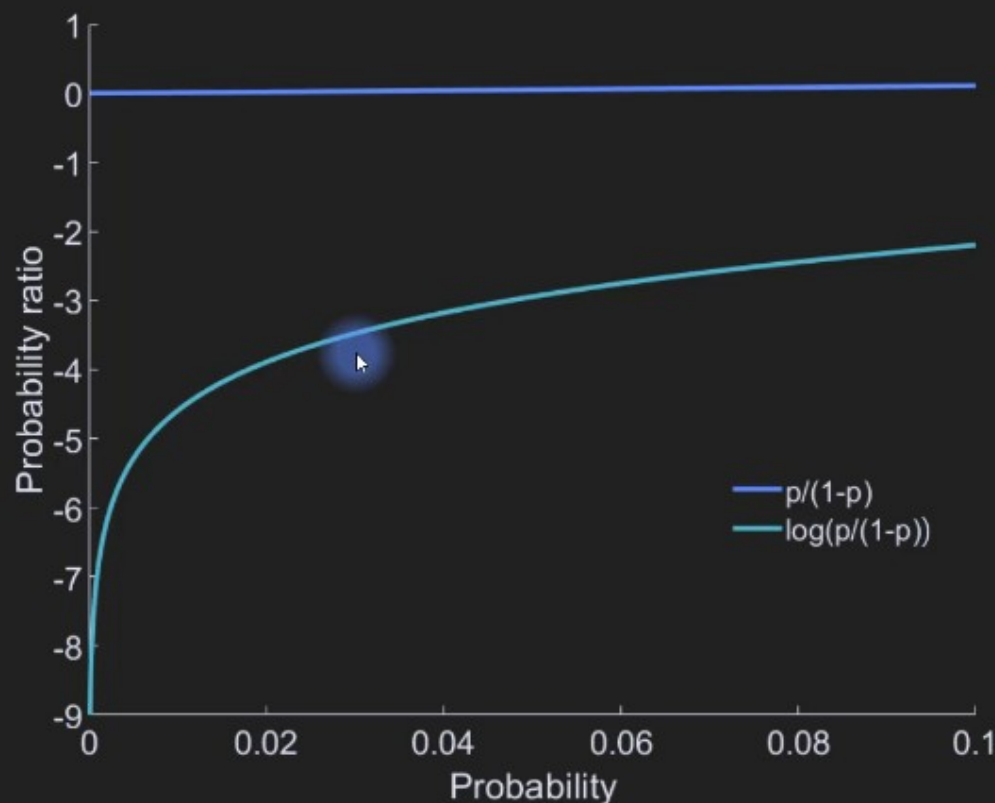
Setting up the logistic regression equation

$$\ln \frac{p}{1-p} = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k$$

$$\frac{p}{1-p} = e^{\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k}$$

$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)}}$$

Why take the log of probabilities?



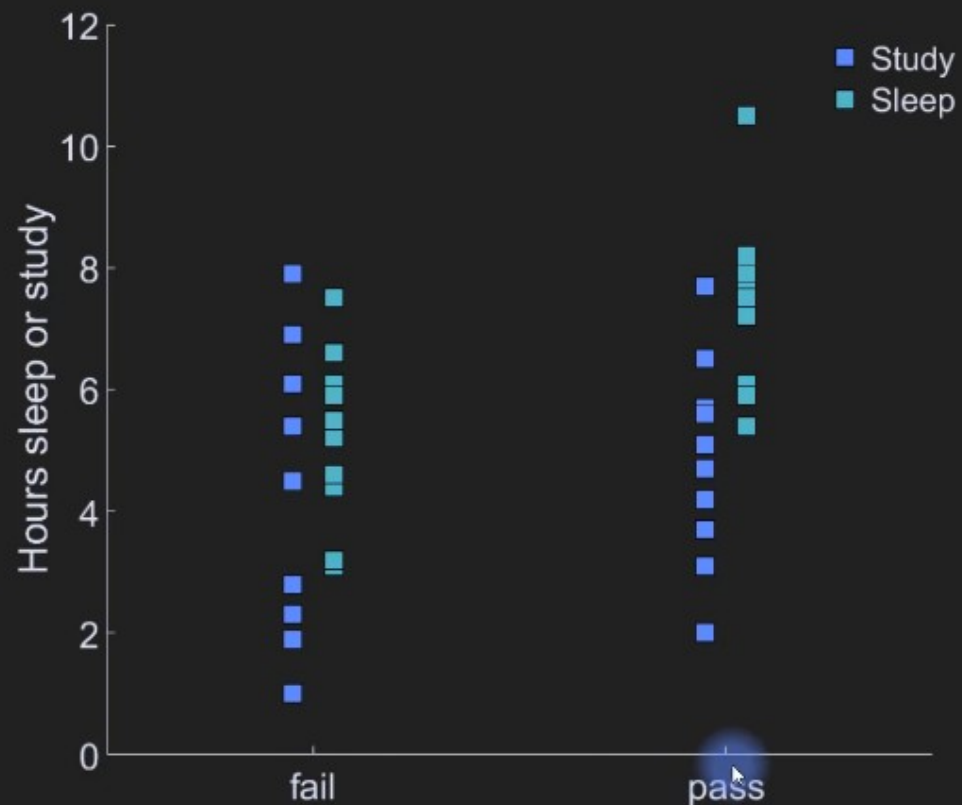
Main point: The log of small values has a larger dynamic range and is easier to work with in optimization problems.

How to find the best regression coefficients?

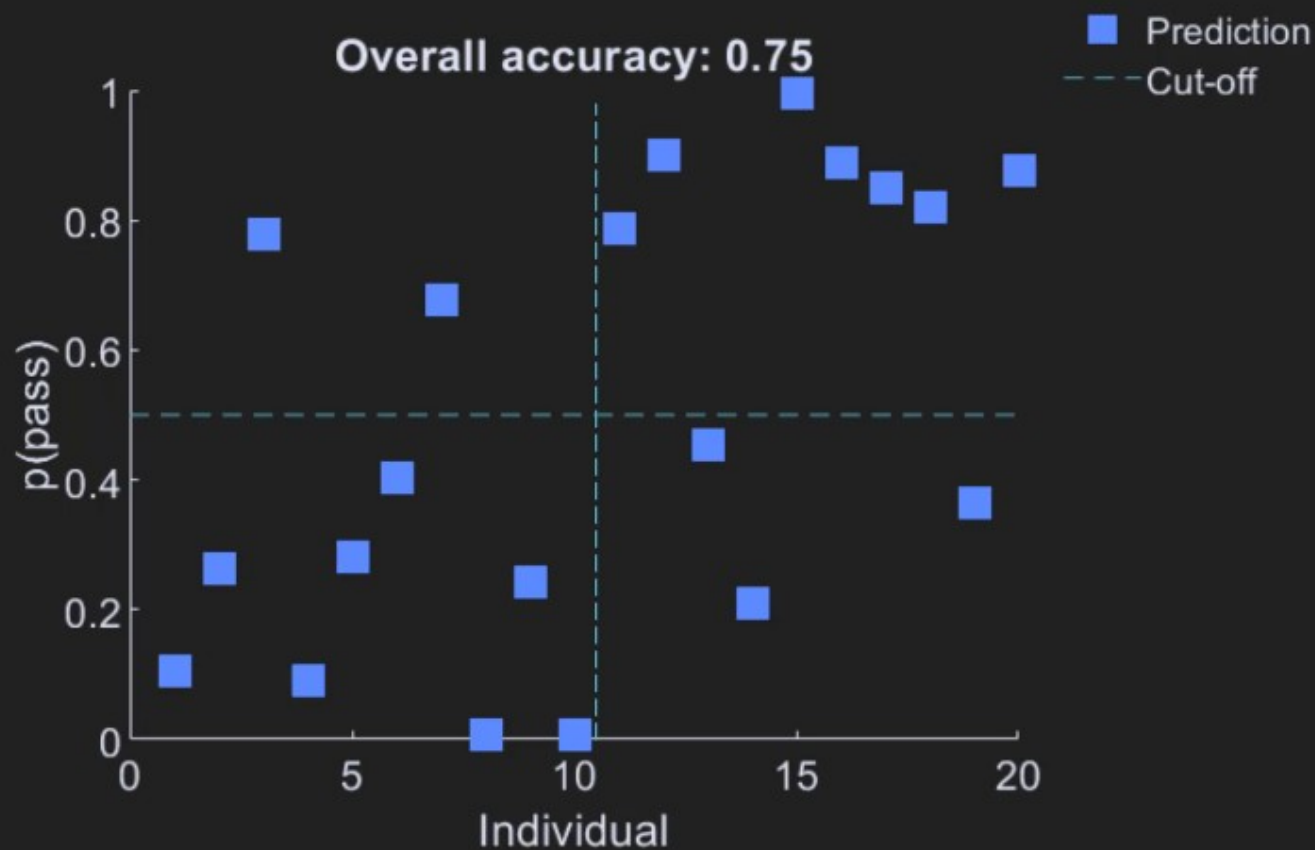
$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)}}$$

The nonlinearities in the coefficients prevent the left-inverse from being a viable solution.

Example logistic regression



Example logistic regression



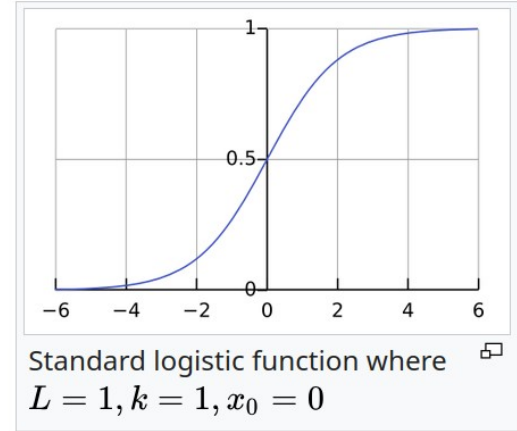
https://en.wikipedia.org/wiki/Logistic_function

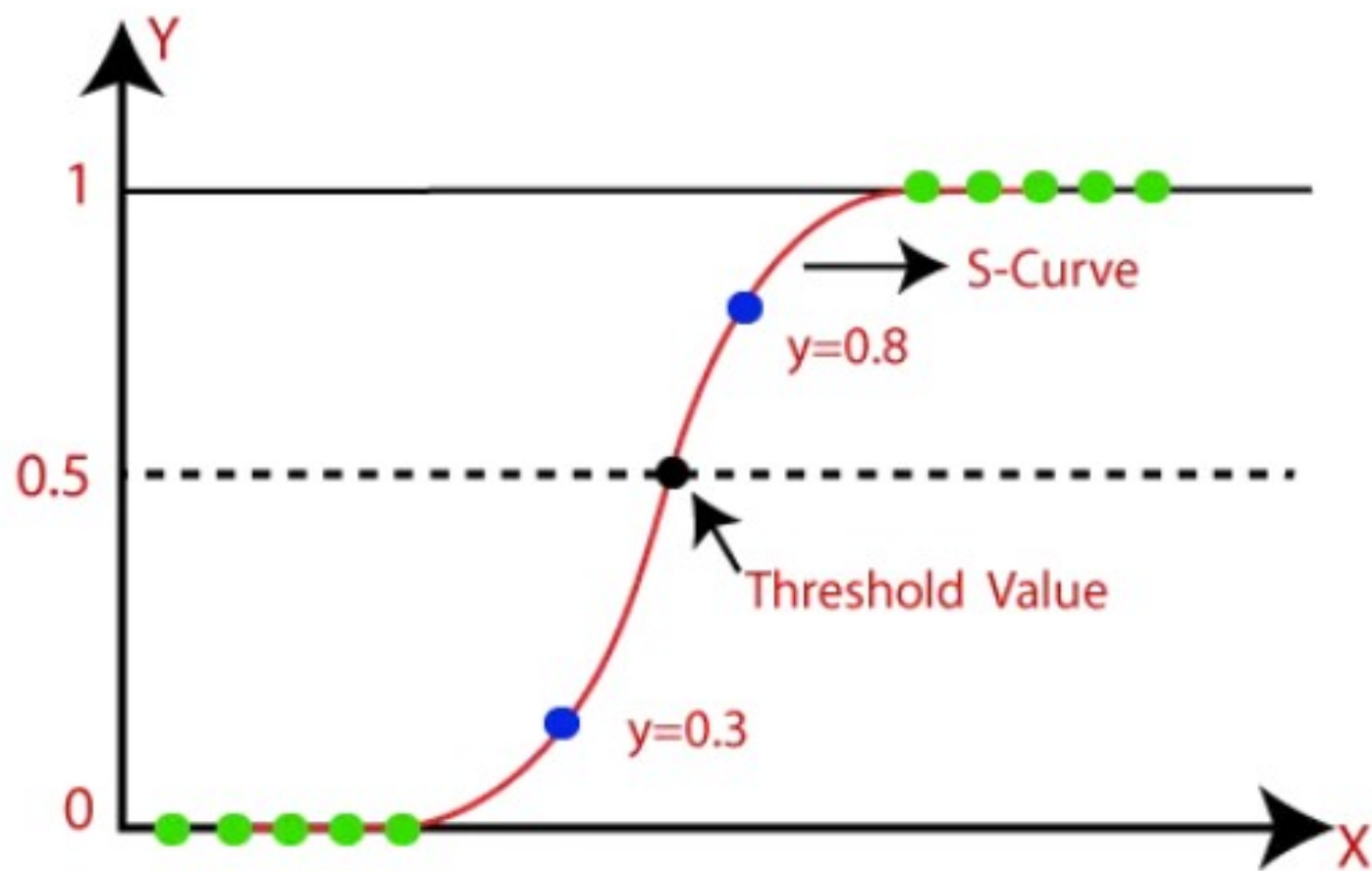
The [exponential function](#) with negated argument (e^{-x}) is used to define the **standard logistic function** where $L = 1$, $k = 1$, $x_0 = 0$, which has the equation

$$f(x) = \frac{1}{1 + e^{-x}}$$

and is sometimes simply called the **sigmoid function**.^[2] It is also sometimes called the **expit**, being the inverse function of the [logit](#).^{[3][4]}

The logistic function finds applications in a range of fields, including [biology](#) (especially [ecology](#)), [biomathematics](#), [chemistry](#), [demography](#), [economics](#), [geoscience](#), [mathematical psychology](#), [probability](#), [sociology](#), [political science](#), [linguistics](#), [statistics](#), and [artificial neural networks](#). There





```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression

## generate the data
exam_outcome = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1];
study_hours = [7.9, 7.9, 2.8, 5.4, 6.1, 4.5, 6.9, 2.3, 1.9, 1, 3.1, 5.7, 5.6, 4.7, 4.2, 2, 7.7, 6.5, 5.1, 3.7]
sleep_hours = [4.4, 5.2, 7.5, 4.6, 5.5, 6.1, 6.6, 3.1, 5.9, 3.2, 7.5, 7.8, 6.1, 5.4, 10.5, 8.2, 7.2, 7.2, 5.9, 7.9]

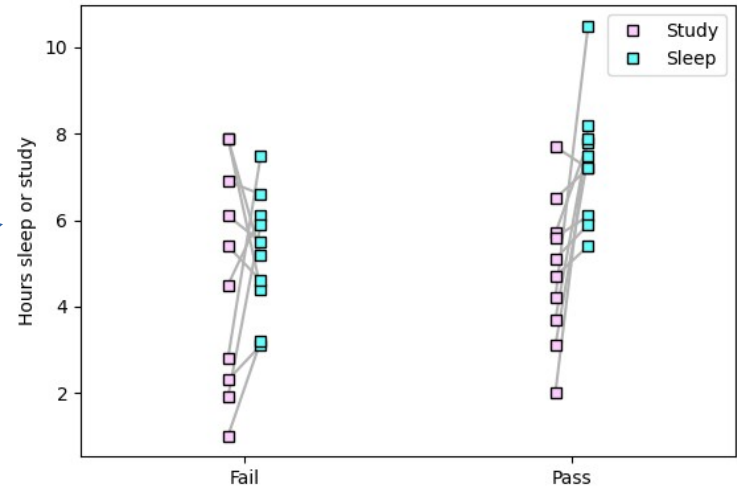
n = len(exam_outcome)

# and plot
for i in range(n):
    plt.plot([exam_outcome[i]-.05, exam_outcome[i]+.05],[study_hours[i],sleep_hours[i]],color=[.7,.7,.7])

plt.plot(exam_outcome-.05*np.ones(n),study_hours,'ks',markerfacecolor=[1,.8,1],label='Study')
plt.plot(exam_outcome+.05*np.ones(n),sleep_hours,'ks',markerfacecolor=[.39,1,1],label='Sleep')

plt.xticks([0,1],labels=('Fail','Pass'))
plt.xlim([-0.5,1.5])
plt.ylabel('Hours sleep or study')
plt.legend()
plt.show()

```



```

## now for the logistic regression
# create a model
logregmodel = LogisticRegression(solver='liblinear')# 'newton-cg')#
# create the design matrix
desmat = np.vstack((study_hours,sleep_hours)).T
logregmodel.fit(desmat,np.array(exam_outcome))
print(logregmodel.intercept_)
print(logregmodel.coef_)

# compute predictions and accuracy
predvals = logregmodel.predict(desmat) # class labels
predvalsP = logregmodel.predict_proba(desmat) # probability values
print(predvals)
print(np.array(exam_outcome))
print(predvalsP)
logregmodel.score(desmat,np.array(exam_outcome))

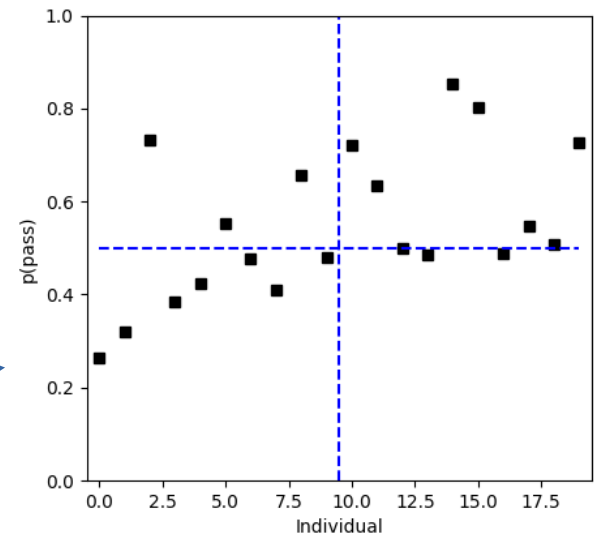
# plotting
fig,ax = plt.subplots(1,1,figsize=(5,5))
ax.plot(predvalsP[:,1],'ks')
ax.plot([0,19],[.5,.5],'-b-')
ax.plot([9.5,9.5],[0,1],'-b-')
ax.set_xlabel('Individual')
ax.set_ylabel('p(pass)')
ax.set_xlim([-0.5, 19.5])
ax.set_ylim([0,1])
plt.show()

```

```

from pandas.core import (
[-0.96510192]
[[-0.19445677  0.3361749 ]]
[0 0 1 0 0 1 0 0 1 0 1 1 0 0 1 1 1 1]
[0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1]
[[0.7353894  0.2646106 ]
[0.67987577 0.32012423]
[0.26664125 0.73335875]
[0.61509116 0.38490884]
[0.5750111  0.4249889 ]
[0.44756611 0.55243389]
[0.52201059 0.47798941]
[0.59150979 0.40849021]
[0.343246   0.656754  ]
[0.5209375  0.4790625 ]
[0.27820281 0.72179719]
[0.36617566 0.63382434]
[0.50084824 0.49915176]
[0.51592069 0.48407931]
[0.1482976  0.8517024 ]
[0.19740089 0.80259911]
[0.51048841 0.48951159]
[0.45229843 0.54770157]
[0.49335028 0.50664972]
[0.27464343 0.72535657]]

```



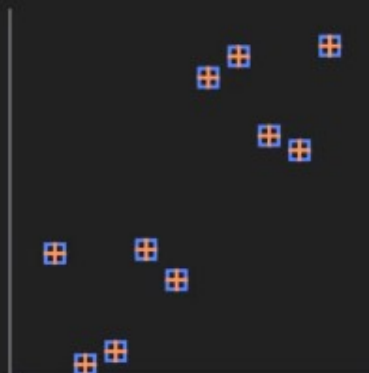
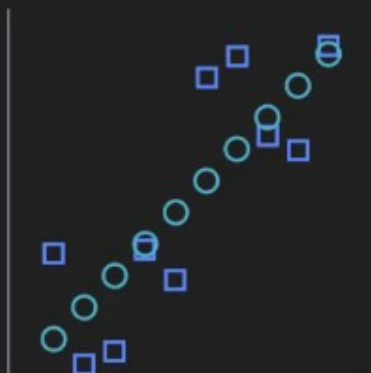
Under- and over-fitting

In this video, you will learn

- What “overfitting” and “underfitting” means.
- The relationship between overfitting and the number of model parameters.
- The hidden and insidious form of overfitting.

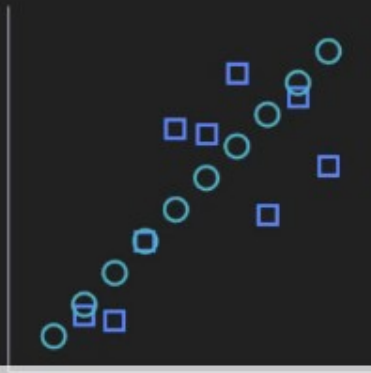
The problem with overfitting

Dataset 1

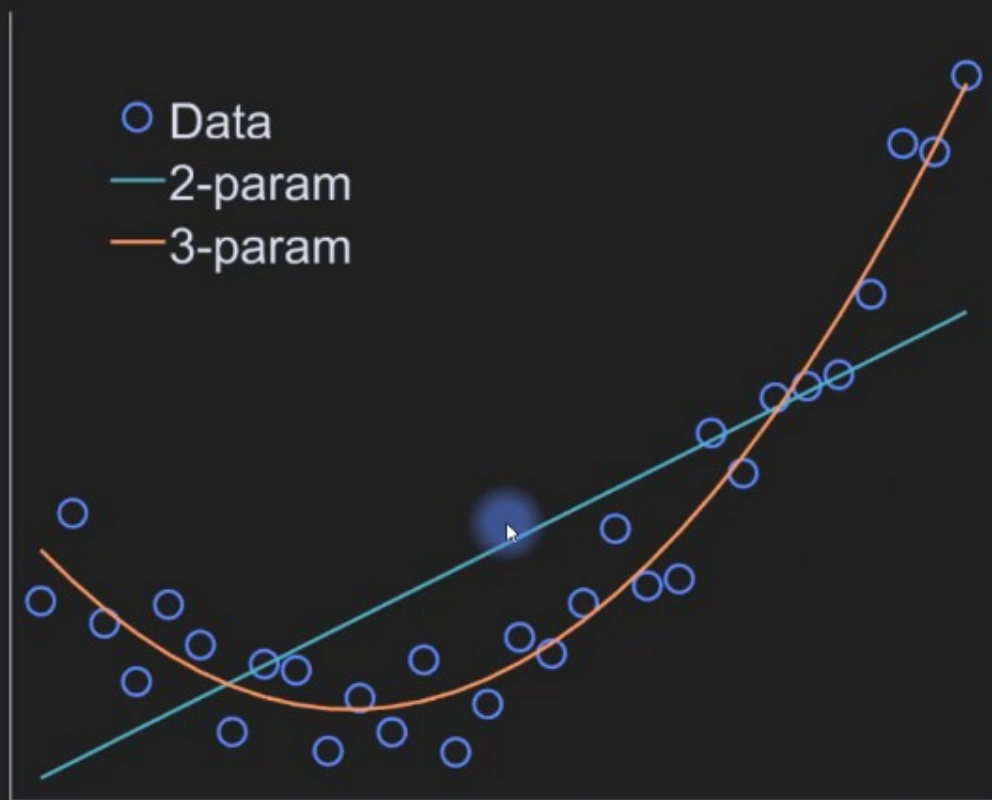


2-parameter model
10-parameter model

Dataset 2



The problem with underfitting



Over- and under-fitting: summary

Overfitting

- Overly sensitive to noise
- Increased sensitivity to subtle effects
- Reduced generalizability
- Over-parameterized models become difficult to estimate

Underfitting

- Less sensitive to noise
- Less likely to detect true effects
- Reduced generalizability
- Parameters are better estimated
- Good results with less data

How to know the correct number of parameters?

With 1-2 dimensions: Visualize the data and make an informed decision.

With 3+ dimensions: Formal model comparisons (next lecture!) (and also the polynomial regression lecture).

— Hidden overfitting: “researcher degrees of freedom” —

“Researcher degrees of freedom”: The researcher has many choices for how to clean, organize, and select the data; and which models and how many models to run.

Example: Test models A, B, and C on the same data. Go back and clean the data again with different criteria, then test the three models again. Publish model B (using $\alpha=.05$) with re-cleaned data.

What is the *actual* p-threshold? $.05 < \alpha < .05*6 = .3$

How to avoid researcher overfitting

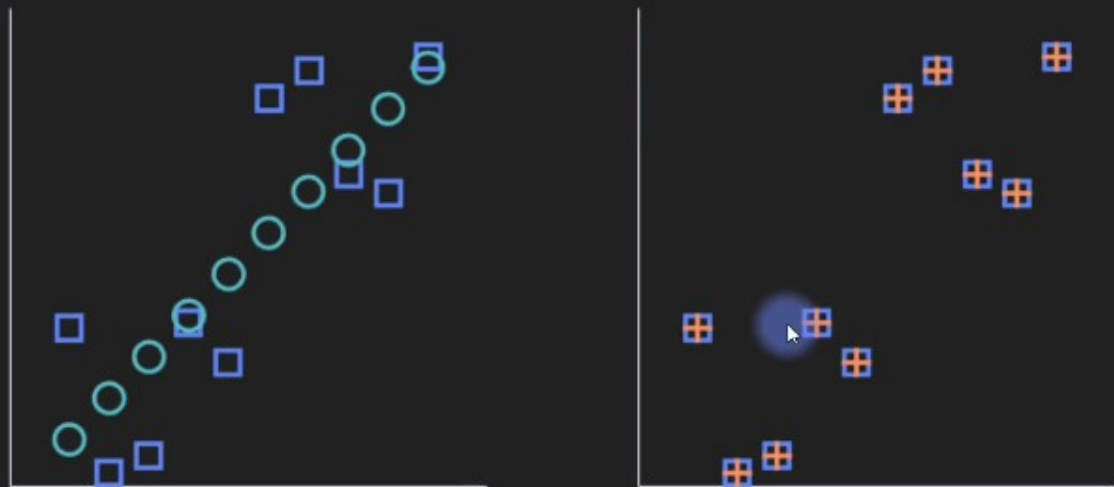
- 1) Decide your complete analysis pipeline in advance (before data collection). Any deviations from this plan must be clearly stated (e.g., pre-registered report).
- 2) Explore/develop an analysis pipeline in a small sample of the data, and then apply that pipeline to the rest of the data. (informal training/testing sets)

Comparing “nested” models

In this video, you will learn

- What “nested” models are.
- How to compare different models of the same data quantitatively.

Blast from the past



Qualitatively, we can see that the wrong number of parameters for a model is bad.

What are “nested” models?

$$M_1 : y = \beta_0 + \beta_1 s + \beta_2 h + \epsilon$$

$$M_2 : y = \beta_0 + \beta_1 s + \beta_2 h + \beta_3 (s \times h) + \epsilon$$

M_2 : “Full model” (extended)

M_1 : “Reduced model” (nested)

What are “nested” models?

$$M_1 : y = \beta_0 + \beta_1 s + \beta_2 h + \epsilon$$

$$M_2 : y = \beta_0 + \beta_1 s + \beta_2 h + \beta_3 (s \times h) + \epsilon$$

A model is “nested” under another model if it contains some identical predictors, and if the DV is the same.

Many models can be nested under the same full model.
Nested models can differ by more than one parameter.

What are “nested” models?

$$M_1 : y = \beta_0 + \beta_1 s + \beta_2 h + \epsilon$$

$$M_2 : y = \beta_0 + \beta_1 s + \beta_2 h + \beta_3 (s \times h) + \epsilon$$

Important observation: The full model will ALWAYS fit the data better than the reduced model. ALWAYS!

But models should be penalized for having more parameters. Thus, more parameters (more complicated model) is justified only when the model significantly improves the fit to the data.

F test for model comparison

$$SS_{\epsilon} = \sum (y_i - \hat{y})^2$$

Sum of squared errors: A measure of the model fit to the data.

$$F(p - k, n - p - 1) = \frac{(SS_{\epsilon}^R - SS_{\epsilon}^F) / (p - k)}{SS_{\epsilon}^F / (n - p - 1)}$$

SS_{ϵ}^F = Full model sum of squared errors

SS_{ϵ}^R = Reduced model sum of squared errors

p = full model parameters

k = reduced model parameters

F test for model comparison

$$SS_{\epsilon} = \sum (y_i - \hat{y})^2$$

Sum of squared errors: A measure of the model fit to the data.

$$F(p - k, n - p - 1) = \frac{(SS_{\epsilon}^R - SS_{\epsilon}^F) / (p - k)}{SS_{\epsilon}^F / (n - p - 1)}$$

F is statistically significant: More parameters improve the model.
Prefer the more complicated model.

F test for model comparison

$$SS_{\epsilon} = \sum (y_i - \hat{y})^2$$

Sum of squared errors: A measure of the model fit to the data.

$$F(p - k, n - p - 1) = \frac{(SS_{\epsilon}^R - SS_{\epsilon}^F) / (p - k)}{SS_{\epsilon}^F / (n - p - 1)}$$

F is statistically significant: More parameters improve the model.
Prefer the more complicated model.

F is nonsignificant: The model with fewer parameters fits as well as the model with more parameters. Prefer the simpler model.

F test for model comparison

$$SS_{\epsilon} = \sum (y_i - \hat{y})^2$$

Sum of squared errors: A measure of the model fit to the data.

$$F(p - k, n - p - 1) = \frac{(SS_{\epsilon}^R - SS_{\epsilon}^F) / (p - k)}{SS_{\epsilon}^F / (n - p - 1)}$$

$$H_0 = \beta_{k+1} = \dots = \beta_p = 0$$

$$H_A = \text{At least one } \beta_{k+1:p} \neq 0$$

What to do about missing data

In this video, you will learn

- What “missing data” means and where it’s gone.
- Four strategies for dealing with missing data.

Missing data

Time 1	Time 2
5	7
6	6
4	?
8	7

Missing data origins:

- Drop-out
- Noise or artifacts
- Mistakes

Missing data, option 1: complete removal

Time 1	Time 2
5	7
6	6
4	?
8	7

Used for:

- Paired data
- Analyzing changes

Missing data, option 3: replacement

Time 1	Time 2
5	7
6	6
4	6.67
8	7

Used for:

- Small datasets

MEDIAN IS BETTER

Replace missing data
with column mean.

Missing data, option 4: prediction

		Time 1	Time 2
Training data		5	7
		6	6
Training data		4	?
		8	7

Testing data

Used for:

- Enough data and other columns (features) to generate a useful predictive model.

